MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory Works No 9 and No 10

Watching System Calls with strace

Adding the trace System Call to xv6

Operating Systems

Student:
Tacu Loredana
FAF-242

Checked by:
Reițman Patricia,
Universitary Assistant

Chișinău 2026

1 Purpose of the Laboratory Works

Laboratory Work No. 9 examines Linux system calls from outside a process with strace. Laboratory Work No. 10 implements a comparable tracing mechanism inside xv6 by adding a trace system call, a per-process mask, kernel-side reporting, and mask inheritance across fork. The completed experiments connect the Linux system-call interface to xv6 kernel dispatch and show how file descriptors, return values, and process state appear at the operating-system boundary.

2 Objectives

1. Observe the openat, read, write, and close calls made by a familiar Linux command. 2.

Write, compile, and trace a C program that uses open, write, read, and close directly. 3.

Explain the purpose of file descriptors 0, 1, 2, and the first dynamically allocated descriptor. 4.

Follow system-call number 23 from an xv6 user program to the kernel handler. 5. Implement

and validate bit-mask-based tracing, including inheritance across fork. 6. Compare external

Linux tracing with tracing implemented inside a small teaching kernel.

3 Theoretical Background

A system call transfers control from a user program to the kernel so that the program can request a protected service. Linux strace observes this boundary and prints each selected call with its arguments and return value. The observed calls in this work include openat for opening a file relative to a directory, read for receiving bytes, write for sending bytes, and close for releasing a descriptor.

Each process starts with descriptor 0 for standard input, descriptor 1 for standard output, and descriptor 2 for standard error. The kernel returns the smallest available descriptor when a new file is opened. Therefore, the first ordinary file normally receives descriptor 3.

xv6 identifies system calls by number. A user stub places the number in register a7 and executes ecall. The kernel dispatcher reads the number, calls the registered handler, and returns the result through a0. The trace mask uses bit N to select system call N, so mask 32 selects bit 5 and therefore SYS_read.

4 Laboratory Work No 9 Watching System Calls with strace

4.1 Tracing a Familiar Command

The first experiment restricted strace to openat, read, write, and close while executing ls -l. The trace was saved separately from the normal directory listing, and the selected calls were counted.

```
strace -f -e trace=openat,read,write,close -o 01_ls_strace.txt ls -l grep -Eo '(openat|read|write|close)\('
01_ls_strace.txt | sort | uniq -c
```

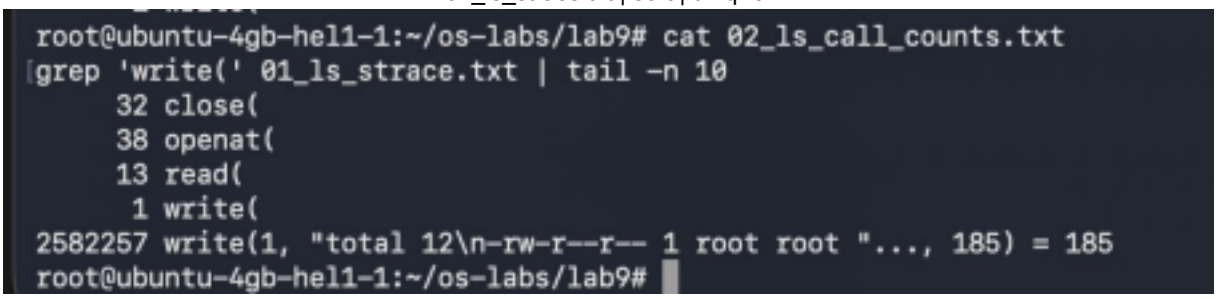


Figure 1 - Filtered system-call counts for ls -l

openat	38	Opened directories, libraries, locale data, and related files
read	13	Read required file and directory data
write	1	Printed the formatted listing to standard output
close	32	Released descriptors that were no longer needed

Observation

The filtered execution contained 84 calls. ls printed through write(1, ...), where descriptor 1 is standard output. The single visible write transferred the formatted listing to the terminal.

4.2 Direct File Operations in C

The second experiment compiled a small program that created note.txt, wrote the line hello, kernel, closed the file, reopened it for reading, and printed the recovered text. strace then exposed the corresponding kernel interactions.

```
gcc -Wall -Wextra -o syscalls syscalls.c
 ./syscalls
strace -e trace=openat,write,read,close -o 07_syscalls_strace.txt ./syscalls grep -E 'note\.txt|hello,
kernel' 07_syscalls_strace.txt
```

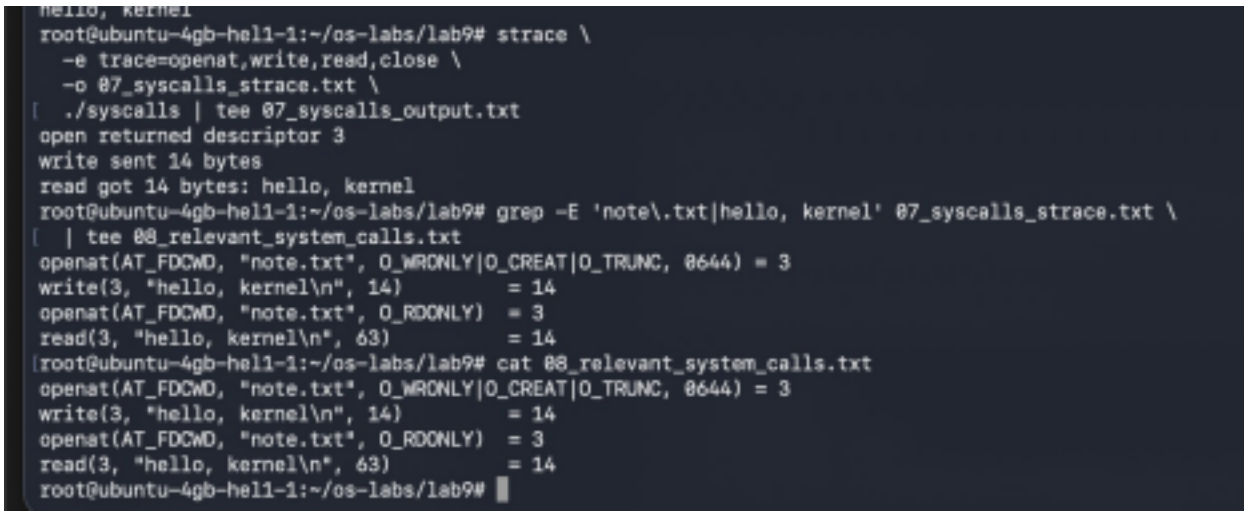


Figure 2 - Program output and the relevant calls for note.txt

Observation

The first openat returned descriptor 3 because descriptors 0, 1, and 2 were already assigned to standard input, standard output, and standard error. write transferred 14 bytes, including the newline, and read returned the same 14 bytes. The successful return values confirm that the requested byte counts were completed.

4.3 Laboratory Work No 9 Results

The experiment showed that even a simple command depends on many kernel services before producing one visible line of output. The smaller C program made the relationship clearer: each source-level file operation appeared as a corresponding system call with a descriptor, byte buffer, byte count, and return value.

5 Laboratory Work No 10 Adding trace to xv6

5.1 Implementation Path

The xv6 implementation followed system-call number 23 from the user interface into the kernel. The user program set a tracing mask and then replaced itself with the requested command through exec. Because the trace mask belongs to the process, it remains active after exec.

kernel/syscall.h	SYS_trace 23	Assigns the system-call number
user/user.h	int trace(int)	Declares the user interface
user/usys.pl	entry trace	Generates the ecall stub
user/trace.c	trace command	Sets the mask and executes another program
kernel/proc.h	trace_mask	Stores the mask per process
kernel/sysproc.c	sys_trace	Copies argument 0 into the current process
kernel/syscall.c	dispatch and printk	Registers and reports selected calls
kernel/proc.c	fork inheritance	Copies the mask into a child process

The syscallnames table maps numbers to readable names. The dispatcher first executes the selected call, then tests (trace_mask >> num) & 1 and prints the PID, name, and value already stored in register a0. Placing the print after the call is essential because the return value is not available earlier.

5.2 Check 1 The Number Reaches the Kernel

Check 1 belongs between the user-side changes and the kernel handler. At that intermediate stage, user/trace.c and the generated ecall stub already pass number 23 in register a7, but the kernel has no handler or dispatch-table entry for it yet. The required diagnostic is therefore:

```
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
```

Observation

This failure is intentional. It proves that the trace user program was included in the xv6 file system, executed ecall with system-call number 23, and reached the kernel dispatcher. Check 2 is performed only after the handler and dispatch-table entry are added.

5.3 Tracing read with Mask 32

trace 32 grep hello README

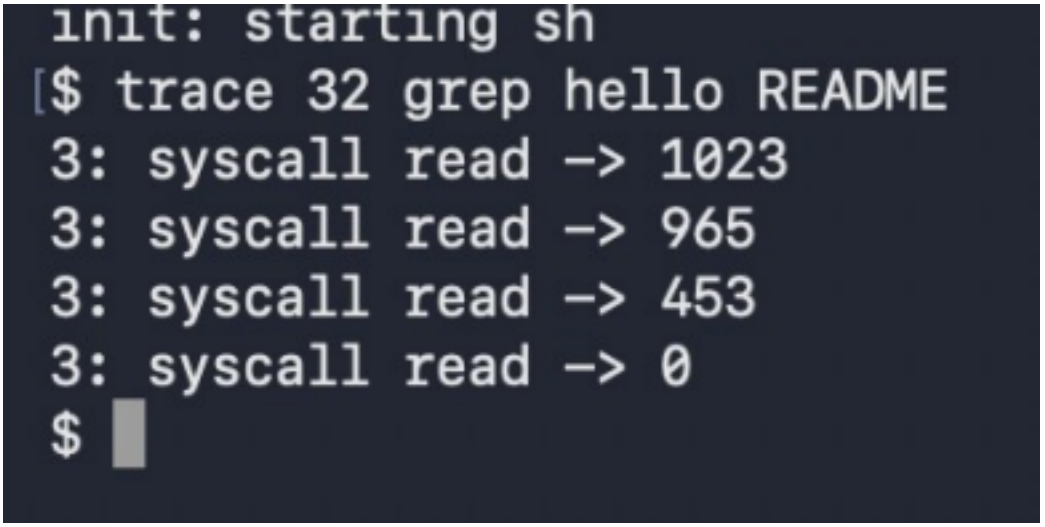


Figure 3 - Successful tracing of SYS_read with mask 32

Observation

Only read appeared because mask 32 equals 2 to the power of 5 and SYS_read is call number 5. The four return values were 1023, 965, 453, and 0. The first three values total 2441 bytes, the size of README, while the final zero marks end of file. The PID may vary between boots without changing the result.

5.4 Tracing All Calls

trace 2147483647 grep hello README

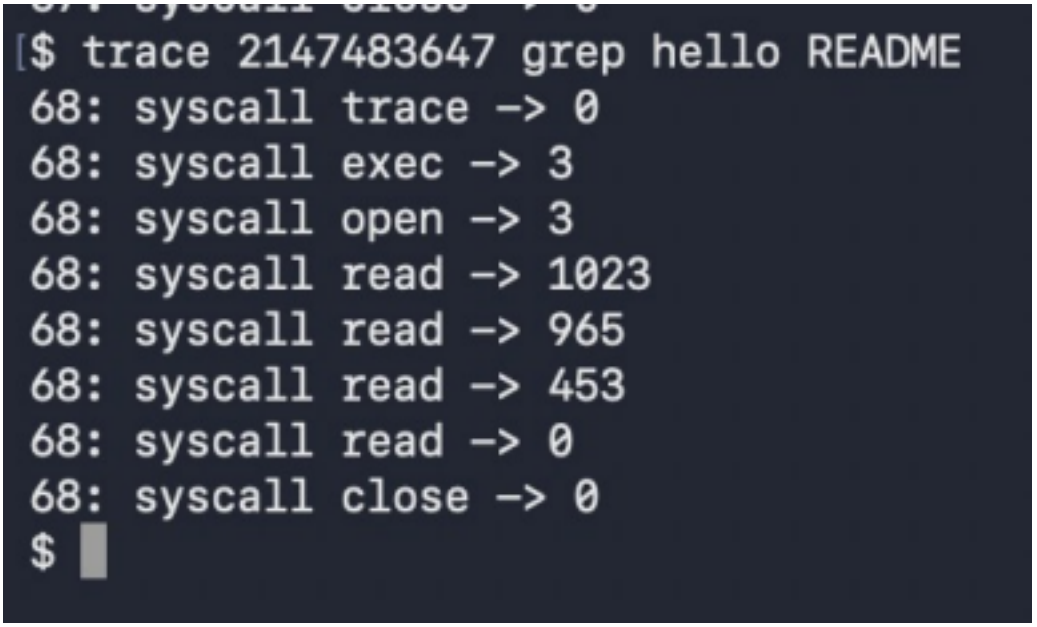


Figure 4 - All enabled system calls for grep

Observation

With bits 0 through 30 enabled, the trace displayed trace, exec, open, four read calls, and close. The sequence shows the command setting its own mask, replacing its program image, opening README, reading until end of file, and closing the descriptor.

5.5 Fork and Write Masks

trace 2 usertests forkforkfork
trace 65536 echo hi
trace 2129920 grep hello README

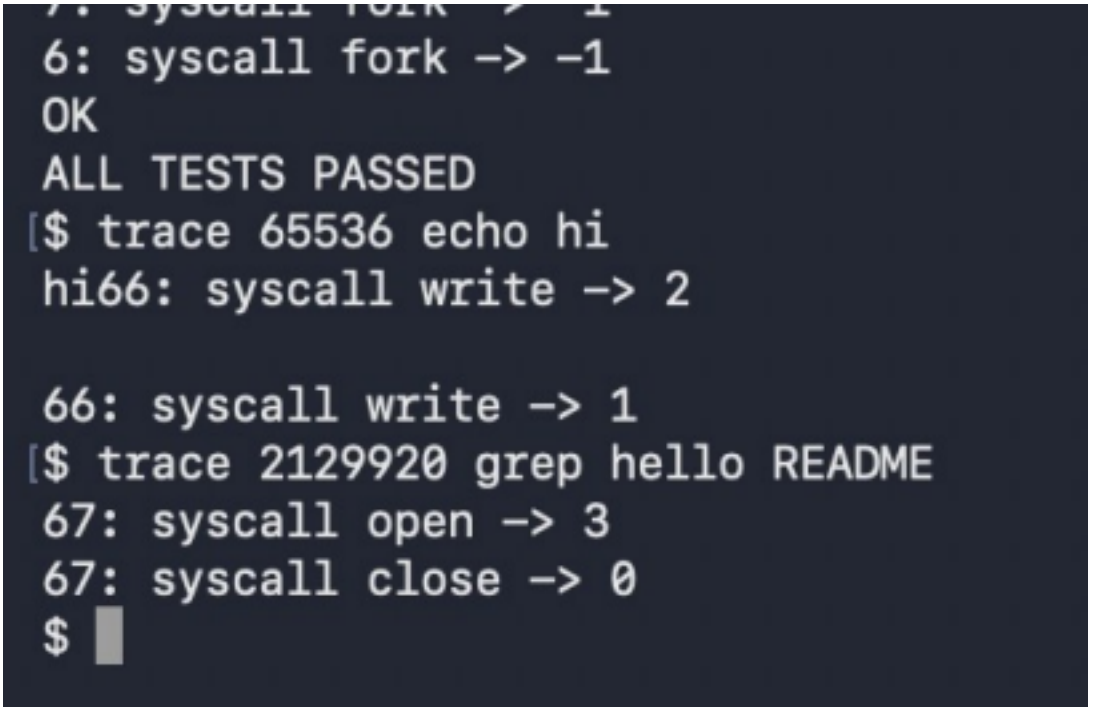


Figure 5 - Fork test, two write calls, and the open-close mask

2	fork	Several PIDs printed fork results and the test passed
65536	write	echo wrote two bytes for hi and one byte for the newline
2129920	open and close	Only open returning 3 and close returning 0 were printed

Observation

The fork test printed lines from several PIDs because fork copies trace_mask into the child through np->trace_mask = p->trace_mask. The echo output placed a trace line between hi and its newline because echo issued two write calls. The mask for open and close is 2^15 + 2^21 = 32768 + 2097152 = 2129920.

5.6 Version Control Evidence

The completed xv6 implementation is recorded on branch lab10-trace. Git history identifies commit 5d3bdd6 with the message Add trace system call for Lab 10. The same commit introduced the system-call number, handler, user program, and Makefile registration.

```
root@ubuntu-4gb-hel1-1:~/xv6-riscv# echo "=== TRACE NUMBER HISTORY ==="
git --no-pager log --oneline -S 'SYS_trace' -- kernel/syscall.h

echo "=== TRACE HANDLER HISTORY ==="
git --no-pager log --oneline -S 'sys_trace' -- kernel/syscall.c kernel/sysproc.c

echo "=== TRACE PROGRAM HISTORY ==="
git --no-pager log --oneline -S 'trace mask command' -- user/trace.c

echo "=== TRACE MAKEFILE HISTORY ==="
git --no-pager log --oneline -S '$U/_trace' -- Makefile

echo "=== CURRENT COMMIT ==="
git --no-pager log -1 --oneline
=== TRACE NUMBER HISTORY ===
5d3bdd6 (HEAD -> lab10-trace) Add trace system call for Lab 10
=== TRACE HANDLER HISTORY ===
5d3bdd6 (HEAD -> lab10-trace) Add trace system call for Lab 10
=== TRACE PROGRAM HISTORY ===
5d3bdd6 (HEAD -> lab10-trace) Add trace system call for Lab 10
=== TRACE MAKEFILE HISTORY ===
5d3bdd6 (HEAD -> lab10-trace) Add trace system call for Lab 10
=== CURRENT COMMIT ===
5d3bdd6 (HEAD -> lab10-trace) Add trace system call for Lab 10
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 6 - Git history for the trace implementation

5.7 Generating and Validating the xv6 Patch

A reusable patch must be exported by Git from the actual xv6 commit. It must not be reconstructed by typing generic hunk headers such as @@ -1 +1,2 @@, because those locations do not identify the real positions in Makefile, struct proc, kfork(), or syscall(). The authentic patch is generated and checked against the parent tree as follows:

```
git show --format= --binary 5d3bdd6 -- Makefile kernel user > xv6-trace.patch git worktree add
/tmp/xv6-before-trace 5d3bdd6^
git -C /tmp/xv6-before-trace apply --check "$(pwd)/xv6-trace.patch" git -C
/tmp/xv6-before-trace apply "$(pwd)/xv6-trace.patch"
make -C /tmp/xv6-before-trace
```

The generated diff contains the real line ranges and surrounding context. In kernel/proc.h, trace_mask is inside struct proc after name. In kernel/proc.c, the mask is initialized in allocproc() and copied inside kfork() immediately after safestrcpy. These locations keep the process state valid and allow the tree to compile with tracing enabled.

6 Comparison of Linux and xv6 Tracing

Location	Runs as an external tracing tool	Implemented in the xv6 kernel
Selection	Names calls with -e trace	Uses a numeric bit mask
Output	Arguments and detailed return values	PID, call name, and return value
Process behavior	Can follow descendants with -f	Mask is copied explicitly during fork
Main benefit	Inspects real Linux programs	Exposes the kernel dispatch mechanism

The two mechanisms observe the same boundary from different positions. strace required no kernel changes and provided detailed arguments for a production operating system. xv6 trace required changes across the user interface, dispatcher, process structure, and fork path, but this smaller implementation made each dependency visible.

7 Testing and Validation

The Linux program compiled with -Wall and -Wextra and produced the expected descriptor and byte counts. The required intermediate Check 1 documents that number 23 reached the kernel before its handler existed. After the kernel changes, the xv6 tree built and booted under QEMU. The read-only mask returned 1023, 965, 453, and 0; the all-calls mask showed the complete open-read-close lifecycle; usertests forkforkfork ended with ALL TESTS PASSED; and the write and open-close masks selected only their intended calls.

The Git history independently confirmed the completed implementation at commit 5d3bdd6. Process IDs differed between executions, which is expected because xv6 assigns them dynamically. The system-call names and return values remained consistent across the recorded tests.

8 Conclusion and Critical Analysis

The experiments demonstrated the system-call boundary at two levels. On Linux, strace revealed that one visible ls output line followed dozens of opens, reads, and closes. The direct C program reduced the trace to a clear file lifecycle and confirmed the standard descriptor convention and byte-count semantics.

The xv6 work showed the internal machinery behind comparable output. A system call needs a stable number, a user stub, a kernel handler, a dispatch-table entry, and process state when its behavior must survive exec or fork. The bit-mask design made selection inexpensive, but it also required careful numbering because the meaning of every bit depends on the system-call table.

All required runtime checks succeeded. The most important implementation detail was mask inheritance in fork: without that assignment, descendants created by forkforkfork would stop reporting calls. The write experiment also illustrated that user-visible text does not necessarily correspond to one write operation; echo sent the text and newline separately. Together, the results provide a consistent explanation of tracing, descriptors, return values, exec, and process inheritance.

Appendix A Linux syscalls Program

```
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>

int main(void) {
 const char *msg = "hello, kernel\n";

 int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);  if (fd < 0) {
perror("open"); return 1; }
 printf("open returned descriptor %d\n", fd);

 ssize_t n = write(fd, msg, strlen(msg));
 printf("write sent %zd bytes\n", n);
 close(fd);

 fd = open("note.txt", O_RDONLY);
 if (fd < 0) { perror("open"); return 1; }
 char buf[64];
 n = read(fd, buf, sizeof(buf) - 1);
 if (n < 0) { perror("read"); close(fd); return 1; }  buf[n] = '\0';
 printf("read got %zd bytes: %s", n, buf);
 close(fd);
 return 0;
}
```

Appendix B xv6 trace User Program

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
 if (argc < 3) {
 fprintf(2, "usage: trace mask command [args...]\n");  exit(1);
 }
 if (trace(atoi(argv[1])) < 0) {
 fprintf(2, "trace: failed\n");
 exit(1);
 }
 exec(argv[2], argv + 2);
 fprintf(2, "trace: exec %s failed\n", argv[2]);  exit(1);
}
```

Appendix C xv6 Kernel Changes

```
/* kernel/syscall.h */
#define SYS_trace 23

/* kernel/proc.h, inside struct proc, after name */ struct proc {
 /* existing fields ... */
 char name[16];
 int trace_mask;
};

/* kernel/sysproc.c */
uint64
sys_trace(void)
{
 int mask;
 argint(0, &mask);
 myproc()->trace_mask = mask;
 return 0;
}

/* kernel/syscall.c */
extern uint64 sys_trace(void);
[SYS_trace] = sys_trace,

p->trapframe->a0 = syscalls[num]();
if ((p->trace_mask >> num) & 1)
 printk("%d: syscall %s -> %ld\n",
 p->pid, syscallnames[num], p->trapframe->a0);

/* kernel/proc.c, inside allocproc() */
p->trace_mask = 0;

/* kernel/proc.c, inside kfork(), after safestrcpy */ safestrcpy(np->name,
p->name, sizeof(p->name)); np->trace_mask = p->trace_mask;
```

Appendix D Commands Used for Validation

```
# Linux
strace -f -e trace=openat,read,write,close ls -l gcc -Wall -Wextra
-o syscalls syscalls.c
 ./syscalls
strace -e trace=openat,write,read,close ./syscalls

# xv6
# Check 1, before adding the kernel handler and table entry trace 32 grep hello
README

# Checks after completing the kernel side
trace 32 grep hello README
trace 2147483647 grep hello README
trace 2 usertests forkforkfork
trace 65536 echo hi
trace 2129920 grep hello README
```